



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF CSE

PROJECT ABSTRACT SUBMISSION

COURSE: COMPILER DESIGN PROJECT EVALUATION (24CS2235F)

A.Y. 2026 – 2027

TITLE OF THE PROJECT	MiniLang: A Multi-Phase Compiler Implementation	
S. No.	University ID	Name
1	2420030638	Rayapureddi Rishi SriCharan
2	2420030161	Yennam Sesank Reddy
3	2420030662	G.Bala Nihith Reddy
Name of the Guide		Dr. Krishna Kishore

Abstract

MiniLang is an educational and practical compiler design project focused on implementing a complete compiler for a small, C-inspired language supporting integer/Boolean variables, expressions, control flow, loops, print statements, and functions. The system processes source code through a full end-to-end pipeline—lexical, syntax, and semantic analysis, intermediate code generation, optimization, and target assembly generation—while outputting descriptive error messages for invalid constructs. Lexical analysis uses regular expressions and finite automata to tokenize code, which an LL(1) or recursive-descent parser validates against a context-free grammar after eliminating left recursion and applying left factoring. Parsing constructs an Abstract Syntax Tree (AST) while building a symbol table to track variable types, scopes, and declarations. Semantic analysis enforces correctness by catching undeclared variables, duplicate declarations, and type mismatches. Valid ASTs are translated into Three-Address Code (TAC) with temporary variables and labels to manage flow control and expressions. An optimization module then refines the intermediate representation using techniques like constant folding, constant propagation, dead-code elimination, common subexpression elimination, DAGs, basic-block optimization, and peephole optimization. Finally, the optimized TAC is converted into assembly-like target code using basic register allocation. To illustrate the complete compiler pipeline, the final system can display outputs from every phase, including tokens, ASTs, symbol tables, intermediate representation, optimized code, and target assembly.

Signature of the Guide

HOD